

PROJECT TITLE

by

STUDENT NAME

URN: 123456/7

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 20XX

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Johann A. Briffa

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Student Name

May 20XX

© Copyright Student Name, May 20XX

Abstract

Write a summary of the work presented in your dissertation. Introduce the topic and highlight your main contributions and results. The abstract should be comprehensible on its own, and should not contain any references. As far as possible, limit the use of jargon and abbreviations, to make the abstract readable by non-specialists in your area. Do not exceed 300 words.

Acknowledgements

Write any personal words of thanks here. Typically, this space is used to thank your supervisor for their guidance, as well as anyone else who has supported the completion of this dissertation, for example by discussing results and their interpretation or reviewing write ups. It is also usual to acknowledge any financial support received in relation to this work.

Contents

1	Introduction	14
1.1	Problem Statement	15
1.2	Motivation	15
1.3	The Challenge of Chess	15
1.3.1	Computational Complexity	15
1.3.2	Failure of Naive Approaches	15
1.4	Critique of Existing Solutions	15
1.5	Overview of My Approach	15
1.5.1	Custom Game Definition Framework	15
1.5.2	Machine Learning Techniques	15
1.5.3	Comparison Methodology	15
1.6	Summary of Key Results	15
1.7	Scope and Limitations	15
1.8	Aims and Objectives	15
1.9	Report Structure	15
2	Literature Review	16
2.1	Classical Chess Engines	16
2.2	Probabilistic Search Methods	16

2.3	The Deep Learning Revolution	16
2.4	Supervised Learning Approaches	16
2.5	Recent Innovations	16
2.6	Game AI Frameworks	16
2.7	Critical Analysis	16
2.8	Summary	16
3	Methodology and Design	17
3.1	Requirements Analysis	17
3.2	Framework Architecture	17
3.2.1	Game Definition Module	17
3.2.2	AI Agent Module	17
3.2.3	Training and Evaluation Engine	17
3.3	Design Decisions and Justification	17
3.4	Chess Implementation	17
3.5	Technical Challenges	17
3.6	Diagrams	17
4	Implementation	18
4.1	Development Environment and Tools	19
4.2	Core Framework Implementation	19
4.3	Supervised Learning Model	19
4.3.1	Data Preparation	19
4.3.2	Neural Network Architecture	19
4.3.3	Training Process	19
4.4	Reinforcement Learning Model	19
4.4.1	AlphaZero-Style Architecture	19

4.4.2	MCTS Implementation	19
4.4.3	Self-Play Engine	19
4.4.4	Training Loop	19
4.5	Baseline Models	19
4.6	Testing and Debugging	19
4.7	Documentation	19
4.8	Sophistication of Knowledge Applied	19
5	Evaluation	20
5.1	Evaluation Methodology	21
5.2	Supervised Learning Model Results	21
5.2.1	Training Performance	21
5.2.2	Playing Strength	21
5.2.3	Error Analysis	21
5.3	Reinforcement Learning Model Results	21
5.3.1	Training Progress	21
5.3.2	Playing Strength	21
5.3.3	Learned Strategy Analysis	21
5.4	Comparative Analysis	21
5.5	Ablation Studies	21
5.6	Comparison with Related Work	21
5.7	Interpretation and Significance	21
5.8	Limitations and Challenges	21
6	Critical Review and Reflection	22
6.1	Achievement of Objectives	22
6.2	What Worked Well	22

6.3	What Didn't Work Well	22
6.4	Lessons Learned	22
6.5	Personal Reflection	22
6.6	Future Work	22
6.6.1	Short-term Extensions	22
6.6.2	Long-term Research Directions	22
6.7	Broader Impact	22
7	Legal, Social, Ethical, and Professional Issues	23
7.1	Ethical Considerations	24
7.1.1	Research Ethics	24
7.1.2	Responsible AI	24
7.2	Legal Issues	24
7.2.1	Intellectual Property	24
7.2.2	Copyright and Licensing	24
7.3	Social Issues	24
7.3.1	Accessibility	24
7.3.2	Impact on Chess Community	24
7.4	Professional Issues	24
7.4.1	Code of Conduct	24
7.4.2	Information Security	24
7.5	Sustainability	24
7.5.1	Environmental Impact	24
7.5.2	UN Sustainable Development Goals	24
7.6	Summary	24
8	Conclusion	25

A	Code Listings	26
B	Additional Results	27
C	User Guide	28
D	Game Examples	29
E	Ethical Approval Documentation	30
F	Project Planning Documents	31

List of Figures

List of Tables

Glossary

A_f	The source message, being a sequence of f source symbols $a_1a_2 \dots a_f$
a_i	The i^{th} symbol in the source message, where $a_i \in S_m$
B_g	The decoded message, being a sequence of g source symbols $b_1b_2 \dots b_g$
b_i	The i^{th} symbol in the decoded message, where $b_i \in S_m$
C_h	The transmitted (compressed) message, being a sequence of h Tunstall codewords $c_1c_2 \dots c_h$
c_i	The i^{th} codeword in the transmitted message, where $c_i \in T_n$
D_h	The received (compressed) message, which for a complete Tunstall code is a sequence of h Tunstall codewords $d_1d_2 \dots d_h$
d_i	The i^{th} codeword in the received message, where $d_i \in T_n$ for a complete Tunstall code

Abbreviations

BER	Bit Error Rate
BPSK	Binary Phase Shift Keying
BSC	Binary Symmetric Channel
DCT	Discrete Cosine Transform
ECC	Error Correcting Codes
FEC	Forward Error Correction
JPEG	Joint Photographic Experts Group
MPEG	Moving Pictures Experts Group
SER	Symbol Error Rate
SNR	Signal to Noise Ratio

Chapter 1

Introduction

1.1 Problem Statement

1.2 Motivation

1.3 The Challenge of Chess

1.3.1 Computational Complexity

1.3.2 Failure of Naive Approaches

1.4 Critique of Existing Solutions

1.5 Overview of My Approach

1.5.1 Custom Game Definition Framework

1.5.2 Machine Learning Techniques

1.5.3 Comparison Methodology

1.6 Summary of Key Results

1.7 Scope and Limitations

1.8 Aims and Objectives

Chapter 2

Literature Review

2.1 Classical Chess Engines

2.2 Probabilistic Search Methods

2.3 The Deep Learning Revolution

2.4 Supervised Learning Approaches

2.5 Recent Innovations

2.6 Game AI Frameworks

2.7 Critical Analysis

2.8 Summary

Chapter 3

Methodology and Design

3.1 Requirements Analysis

3.2 Framework Architecture

3.2.1 Game Definition Module

3.2.2 AI Agent Module

3.2.3 Training and Evaluation Engine

3.3 Design Decisions and Justification

3.4 Chess Implementation

3.5 Technical Challenges

3.6 Diagrams

Chapter 4

Implementation

4.1 Development Environment and Tools

4.2 Core Framework Implementation

4.3 Supervised Learning Model

4.3.1 Data Preparation

4.3.2 Neural Network Architecture

4.3.3 Training Process

4.4 Reinforcement Learning Model

4.4.1 AlphaZero-Style Architecture

4.4.2 MCTS Implementation

4.4.3 Self-Play Engine

4.4.4 Training Loop

4.5 Baseline Models

4.6 Testing and Debugging

4.7 Documentation

Chapter 5

Evaluation

5.1 Evaluation Methodology

5.2 Supervised Learning Model Results

5.2.1 Training Performance

5.2.2 Playing Strength

5.2.3 Error Analysis

5.3 Reinforcement Learning Model Results

5.3.1 Training Progress

5.3.2 Playing Strength

5.3.3 Learned Strategy Analysis

5.4 Comparative Analysis

5.5 Ablation Studies

5.6 Comparison with Related Work

5.7 Interpretation and Significance

Chapter 6

Critical Review and Reflection

6.1 Achievement of Objectives

6.2 What Worked Well

6.3 What Didn't Work Well

6.4 Lessons Learned

6.5 Personal Reflection

6.6 Future Work

6.6.1 Short-term Extensions

6.6.2 Long-term Research Directions

6.7 Broader Impact

Chapter 7

Legal, Social, Ethical, and Professional Issues

7.1 Ethical Considerations

7.1.1 Research Ethics

7.1.2 Responsible AI

7.2 Legal Issues

7.2.1 Intellectual Property

7.2.2 Copyright and Licensing

7.3 Social Issues

7.3.1 Accessibility

7.3.2 Impact on Chess Community

7.4 Professional Issues

7.4.1 Code of Conduct

7.4.2 Information Security

7.5 Sustainability

Chapter 8

Conclusion

Appendix A

Code Listings

Appendix B

Additional Results

Appendix C

User Guide

Appendix D

Game Examples

Appendix E

Ethical Approval Documentation

Appendix F

Project Planning Documents

Bibliography

- Briffa, J. A., Schaathun, H. G. & Wesemeyer, S. (2010), An improved decoding algorithm for the Davey-MacKay construction, *in* ‘Proc. IEEE Intern. Conf. on Commun.’, Cape Town, South Africa.
- Burrows, M. & Wheeler, D. J. (1994), A block-sorting lossless data compression algorithm, Technical report, Digital SRC Research Report.
- el Gamal, A. A., Hemachandra, L. A., Shperling, I. & Wei, V. K. (1987), ‘Using simulated annealing to design good codes’, *IEEE Transactions on Information Theory* **33**(1), 116–123.
- Farrell, P. G. (1992), ‘Notes on source coding’. University of Manchester.
- Henderson, B. (2009), *Netpbm*.
URL: <http://netpbm.sourceforge.net/doc/>
- NVI (2009), *NVIDIA CUDA Programming Guide*. Version 2.3.
- Press, W. H., Teukolsky, S. A., Vetterling, W. T. & Flannery, B. P. (1992), *Numerical Recipes in C: The Art of Scientific Computing*, second edn, Cambridge University Press.
- Tunstall, B. P. (1968), Synthesis of Noiseless Compression Codes, PhD thesis, Georgia Institute of Technology.